

UASLP
Facultad de ingeniería

Área: Carrera:

CICOMP Computación

Nombre y Clave del alumno:

Jacqueline Adriana Lozoya Silos
321979

Materia:

Fundamentos de Desarrollo Móvil

Tarea:

Documento uso de IA durante el examen

Fecha de entrega:

14/09/26

Semestre 2026 – 2027



Uso de ia

Prompt usado:

dime como podria implementar y una breve explicacion de lo siguiente, para una aplicacion de flutter:

- Propiedad actions en el AppBar: Investiga cómo usar esta propiedad del AppBar para colocar un IconButton alineado a la esquina superior derecha (para el botón que lleva al perfil).
- Widget CircleAvatar: Investiga cómo utilizar este widget para mostrar un icono o una imagen circular (para la foto de perfil del usuario).
- Widget Divider: Descubre cómo colocar este widget para trazar una línea horizontal sutil que separe el nombre del usuario de sus tarjetas de estadísticas.
- Propiedad MainAxisAlignment.spaceEvenly (o spaceAround): Investiga cómo aplicar estas propiedades dentro de un Row para que tus tarjetas de estadísticas queden perfectamente centradas y distribuidas horizontalmente.

Respuesta:

Aquí tienes cómo implementar cada uno, pensando en una pantalla de perfil típica:

1. actions en el AppBar

La propiedad actions recibe una lista de widgets que Flutter alinea automáticamente a la derecha del AppBar, después del título.

```
AppBar(  
  title: Text('Inicio'),  
  actions: [  
    IconButton(  
      icon: Icon(Icons.person),  
      onPressed: () {  
        Navigator.push(  
          context,  
          MaterialPageRoute(builder: (context) => PerfilScreen()),  
        );  
      },  
    ),  
  ],  
)
```

Si necesitas varios íconos (por ejemplo notificaciones + perfil), simplemente agregas más IconButton a la lista; se acomodan en orden de izquierda a derecha dentro de esa zona.

2. CircleAvatar

Recorta su contenido en un círculo. Puede mostrar una imagen, un ícono, o iniciales como texto.

// Con imagen de red o de assets

```
CircleAvatar(  
  radius: 40,
```

```
    backgroundImage: NetworkImage('https://miapp.com/foto.jpg'),
  )
```

// Con ícono como placeholder (útil si el usuario no tiene foto)

```
CircleAvatar(
  radius: 40,
  backgroundColor: Colors.grey[300],
  child: Icon(Icons.person, size: 40, color: Colors.grey[700]),
)
```

// Con iniciales

```
CircleAvatar(
  radius: 40,
  child: Text('JC', style: TextStyle(fontSize: 20)),
)
```

3. Divider

Es una línea horizontal delgada que ocupa el ancho disponible; se coloca directamente en el flujo de una Column, no requiere un Row contenedor.

```
Column(
  children: [
    Text('Juan Pérez', style: TextStyle(fontSize: 20, fontWeight:
FontWeight.bold)),
    SizedBox(height: 12),
    Divider(
      thickness: 1,
      color: Colors.grey[300],
      indent: 20, // margen desde el borde izquierdo
      endIndent: 20, // margen desde el borde derecho
    ),
    SizedBox(height: 12),
    // aquí irían las tarjetas de estadísticas
  ],
)
```

thickness controla el grosor de la línea, y indent/endIndent te dan márgenes laterales para que no toque los bordes de la pantalla.

4. MainAxisAlignment.spaceEvenly / spaceAround

Se aplica en la propiedad mainAxisAlignment de un Row, para distribuir el espacio entre los hijos a lo largo del eje horizontal.

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: [
    TarjetaEstadistica(titulo: 'Turnos', valor: '12'),
    TarjetaEstadistica(titulo: 'Horas extra', valor: '5'),
    TarjetaEstadistica(titulo: 'Faltas', valor: '1'),
  ],
)
```

Diferencia clave entre las dos opciones:

- **spaceEvenly**: reparte espacio igual entre *todos* los huecos, incluyendo antes del primer elemento y después del último. Da el look más "centrado y equilibrado".
- **spaceAround**: cada elemento tiene el mismo espacio a su alrededor, pero el espacio en los bordes (antes del primero y después del último) es la mitad que el espacio entre elementos.

Para tarjetas de estadísticas, spaceEvenly suele verse más simétrico visualmente; spaceAround es mejor si quieres que las tarjetas se sientan "pegadas" un poco más a los bordes de la pantalla.

Realice otra investigación para recordar el uso del botón nativo de regreso y se me contesto: el "botón nativo para regresar" **no lo escribes tú** — Flutter lo agrega solo cuando detecta que la pantalla actual no es la raíz de la navegación (es decir, fue alcanzada con push, no es la home: de MaterialApp). Por lo que no cree el botón como en clase y deje solo el que da por default